Cambridge (CIE) A Level Computer Science

Your notes

Floating-point Numbers, Representation & Manipulation

Contents

- Floating-Point Basics
- Floating-Point Conversions
- Precision Issues

© 2025 Save My Exams, Ltd.

Your notes

Floating-point binary

What is floating-point binary?

- Floating-point binary is a method of **representing real numbers in binary**
- Including **fractions** and **very large or small values**
- By using **scientific notation in base-2**
- Unlike fixed-point binary (which has a fixed position for the binary point)
  - Floating-point binary allows the point to **"float"** by using a **mantissa** and an **exponent**
- This allows:
  - Representation of **very large and very small numbers**
  - **More precision** for fractional values
  - Efficient use of **limited storage** (e.g. 8-bit, 16-bit)

Components of floating-point

Component	Description
Mantissa	Holds the **significant digits** of the number (like the digits in scientific notation)
Exponent	Indicates how far to **move the binary point**, by powers of 2

- For example, in decimal:

3.14×10^3
Mantissa = 3.14
Exponent = 3

- In binary, floating point works the same way, but uses **powers of 2**:

$1.11 \times 2^4 \rightarrow$ Mantissa: 1.11, Exponent: 4

Representation in binary

- A floating-point binary number typically includes:
  - A **sign bit**
  - A **mantissa** (binary digits with a fixed or implied binary point)
  - An **exponent** (with a sign, often using two's complement)
- Example using 8 bits:

© 2025 Save My Exams, Ltd.

0 1001 0110
[Sign] [Exponent] [Mantissa]

Your notes

Positive floating-point representation

- The **MSB** (most significant bit) is 0 → positive number
- The exponent **shifts the binary point right or left**
- Allows **precision with fractional values**
- Benefit: Increased **precision**
- Limitation: **Lower maximum value** compared to integer binary

Negative floating-point representation

- **Two's complement** is used for negative values
- The MSB of the exponent or mantissa is **used to indicate sign**
- The binary point is **shifted using the** (possibly negative) **exponent**

Example: Representing 6.25 in binary

- **Step 1**: Convert the integer part (6) to binary

6 in binary = 110

- **Step 2**: Convert the fractional part (0.25) to binary
  - Multiply by 2, keep the integer part, and repeat with the remainder:

0.25 × 2 = 0.50 → 0
0.50 × 2 = 1.00 → 1

- **So**

0.25 in binary = 01

- **Step 3**: Combine integer and fractional parts

6.25 in binary = 110.01

Normalising floating-point numbers

- A floating point binary number is said to be **normalised** when the **mantissa begins with 01 (for positive numbers)** or **10 (for negative numbers)**
- This format ensures:
  - **Maximum precision** from the available bits
  - A **consistent standard** for binary arithmetic and comparisons

Why normalise?

Reason	Explanation

Consistency	Ensures all numbers follow the same structure
Precision	Removes unnecessary leading 0s to make full use of the mantissa
Easier processing	Simplifies comparison and arithmetic operations

Your notes

Steps to normalise a floating-point number

1. **Shift the binary point** until the mantissa starts with **01** (positive) or **10** (negative)
2. **Adjust the exponent** each time you move the binary point:
   - Moving the point **left** → **increase** the exponent
   - Moving the point **right** → **decrease** the exponent

Example

- **Before normalisation**
  - Mantissa: 0.0011
  - Exponent: 0010 (which is +2 in binary)
- **Process**
  - Shift binary point **2 places right** to make mantissa start with 01
    - Mantissa becomes: 0.1100
    - Exponent is reduced by 2 → becomes: 0000 (which is 0 in binary)
- **After normalisation**
  - Mantissa: 0.1100
  - Exponent: 0000

© 2025 Save My Exams, Ltd.

Your notes

Denary to floating-point binary

How do you convert denary to floating-point binary?

- For the following conversions we are assuming:
  - 12-bit **mantissa** (in two's complement)
  - 4-bit **exponent** (in two's complement)

Convert +5.25 to floating-point binary

- **Step 1**: Convert 5.25 to binary

5 = 101
0.25 = 0.01
→ 5.25 in binary = 101.01

- **Step 2**: Normalise (form 1.xxxxx × 2^n)

101.01 → 1.0101 × 2^2
→ Mantissa (before padding): 1.0101

- **Step 3**: Remove leading 1 and pad to 12 bits

Mantissa = 010100000000
(1.0101 → drop leading 1, pad with 0s to 12 bits)

- **Step 4**: Write exponent in 4-bit two's complement

Exponent = 2 → 0010

- Final answer

Mantissa	Exponent
010100000000	0010

Convert −5.25 to floating-point binary

- **Step 1**: Convert +5.25 to normalised form

+5.25 = 1.0101 × 2^2 → mantissa = 010100000000

- **Step 2**: Convert mantissa to two's complement (negative)
  - Take the **12-bit** two's complement of 010100000000:
    - Invert: 101011111111
    - Add 1: 101100000000
  - Mantissa for −5.25 = 101100000000

© 2025 Save My Exams, Ltd.

- **Step 3**: Exponent is the same (+2)
  - Two's complement of +2 in 4-bit binary = 0010
- Final answer

Mantissa	Exponent
101100000000	0010

Summary table

Value	Binary	Normalised	Mantissa (12-bit)	Exponent (4-bit)
+5.25	101.01	1.0101×2^2	010100000000	0010
-5.25	−101.01	-1.0101×2^2	101100000000	0010

Floating-point binary to denary

How do you convert floating-point binary to denary?

- For the following conversions we are assuming:
  - 12-bit **mantissa** (in two's complement)
  - 4-bit **exponent** (in two's complement)

Convert the following to denary (+)

Mantissa: 010100000000
Exponent: 0010

- **Step 1**: Convert exponent from 4-bit two's complement to denary

0010 (binary) = +2 (denary)

- **Step 2**: Add back the implicit binary point in the mantissa
  - Mantissa is stored without the leading 1 (because it's normalised)
  - So:

Stored mantissa: 010100000000
Restore: 1.01000000000

- **Step 3**: Shift the binary point according to the exponent
  - Since exponent = +2 → shift binary point **2 places to the right**:

1.01000000000 → 101.000000000

- **Step 4**: Convert binary to denary

Your notes

101 = 5
0.000000000 = 0

Final result: 5.0 (denary)

Convert the following to denary (-)

Mantissa: 101100000000
Exponent: 0010

- **Step 1**: Exponent is again 0010 → +2
- **Step 2**: Convert 12-bit two's complement mantissa to denary:
  - 101100000000 = **negative value** (MSB = 1)
  - Take two's complement to find magnitude:
    - Invert: 010011111111
    - Add 1: 010100000000 → which is 1.01000000000
    - Convert: 101.0 → **-5.0**
- Final result: **-5.0 (denary)**

Summary table

Mantissa	Exponent	Result (Denary)
010100000000	0010	+5.0
101100000000	0010	-5.0

Worked Example

Numbers are stored in two different computer systems by using floating-point representation.

System 1 uses:

- 10 bits for the mantissa
- 6 bits for the exponent
- two's complement form for both the mantissa and the exponent.

System 2 uses:

- 8 bits for the mantissa
- 8 bits for the exponent
- two's complement form for both the mantissa and the exponent.

Calculate the normalised floating-point representation of 113.75 and show how it would be represented in each of these two systems.

Your notes

Show your working. **[4]**

Answer

- conversion of 113.75 to binary seen 1110001.11 [1 mark]
- exponent for normalisation 7 converted to binary 111 **OR** evidence of binary [1 mark]
- point moved 7 places **OR** evidence of finding exponent = 7 [1 mark]
- system 1 answer [1 mark]
- system 2 answer showing correct version from system 1 [1 mark]

System 1

Mantissa	Exponent
0111000111	000111

System 2

Mantissa	Exponent
01110001	00000111

Your notes

Precision Issues

Your notes

Approximation & rounding

- In computing, **real numbers** are often stored using **floating-point binary**, which has a **limited number of bits** for the mantissa and exponent
- This means not all real numbers can be represented **exactly**, only **approximated**

Rounding errors

- Some decimal values cannot be **precisely represented** in binary
- **Example**:

Decimal: 0.1
Binary: 0.000110011001100... (recurring)

- Because memory is limited (e.g. 32 or 64 bits), the binary value must be **truncated or rounded**, which introduces **small errors**
- **Consequence**:
  - Calculations using approximated values may **accumulate errors**
  - Final results may be **slightly inaccurate**

Underflow

- Underflow occurs when a number is **too close to zero** to be stored in the available number of bits
- **Example**:
  - A very small number like 1.2×10^{-40}
  - The exponent is too negative to be stored
  - Result: **Stored as 0**
- **Consequence:**
  - Loss of precision
  - Can affect accuracy in scientific or financial calculations

Overflow

- **Overflow** occurs when a number is **too large** to fit in the allocated bits for the exponent or mantissa
- **Example:**
  - A calculation produces 1.2×10^{50}, but the maximum representable number is 1.2×10^{38}
- **Consequence:**
  - Program may crash or return an **infinity**, **error**, or **wraparound value**

© 2025 Save My Exams, Ltd.

- Can be especially dangerous in loops or financial applications

Summary table

Issue	Cause	Effect
Rounding error	Real number can't be exactly represented in binary	Slight inaccuracies in results
Underflow	Number too small to be stored (exponent too negative)	Value becomes 0
Overflow	Number too large to be stored (exponent too positive)	Incorrect result or program error

Your notes

© 2025 Save My Exams, Ltd.